Public Jump Host and Private RHEL Instance on AWS

Practical Lab Guide

1 Purpose and Learning Outcomes

This practical guide demonstrates a controlled administrative access pattern in AWS. A Windows workstation connects to a public Ubuntu EC2 instance, which acts as a jump host. From that host, an administrator connects over the VPC's private network to a Red Hat Enterprise Linux (RHEL) EC2 instance that has no public IPv4 address.

By the end of the exercise, the learner should be able to:

1. identify a VPC and plan a non-overlapping subnet CIDR;
2. associate a dedicated route table with a private subnet;
3. distinguish routing decisions from security-group permissions;
4. configure a public jump host and a private target instance;
5. use PuTTY, PuTTYgen, Pageant, and SSH public-key authentication;
6. create Linux users and configure `authorized_keys`;
7. inspect ownership, permission bits, and SELinux contexts; and
8. validate the connection path and troubleshoot common failures.

2 Scope, Assumptions, and Prerequisites

2.1 Scope

This is a reusable procedure, not a record tied to one AWS account. Replace every value written as `<VALUE>` with the value from your own environment. The screenshots supplied with this guide document one example run and may therefore display different, account-specific values. Treat screenshots as illustrations of the interface, not as values to copy blindly.

The networking concepts in this guide are included as independent-study background. Their inclusion does not imply that they have already been covered in a formal course.

2.2 Required access and software

- An AWS account with permission to manage EC2 instances, VPC subnets, route tables, and security groups.
- A Windows workstation with PuTTY, PuTTYgen, and Pageant installed.
- A public Ubuntu EC2 instance and a RHEL EC2 instance, preferably in the same VPC.
- An initial administrative SSH path to the Ubuntu instance and an authorized administrative path to RHEL.
- A region selected consistently throughout the exercise.

2.3 Record the environment before making changes

Record the following values in your own lab notes. Do not publish account identifiers, public IPs, or key material unnecessarily.

Value to record	Your value
AWS region	`<AWS_REGION>`
VPC ID and IPv4 CIDR	`<VPC_ID>`, `<VPC_CIDR>`
Public subnet and Availability Zone	`<PUBLIC_SUBNET>`, `<AZ>`
Public Ubuntu instance ID	`<UBUNTU_INSTANCE_ID>`
Ubuntu private and current public IPv4	`<UBUNTU_PRIVATE_IP>`, `<UBUNTU_PUBLIC_IP>`
Private subnet CIDR	`<PRIVATE_SUBNET_CIDR>`
RHEL instance ID and private IPv4	`<RHEL_INSTANCE_ID>`, `<RHEL_PRIVATE_IP>`
Public and private security groups	`<PUBLIC_SG_ID>`, `<PRIVATE_SG_ID>`
Linux usernames	`<UBUNTU_USER>`, `<RHEL_USER>`

2.4 Cost and safety checks

- Check expected EC2, storage, public IPv4, and NAT Gateway charges before creating resources.
- Never place private-key contents, passwords, or passphrases in this report, a repository, or a chat.
- Restrict SSH to a trusted client address on the public host. On the private host, permit SSH only from the jump host's security group.
- Do not delete or modify existing networking resources until you have confirmed which resources the other instances depend on.

3 Conceptual Background

3.1 VPC and CIDR

A Virtual Private Cloud (VPC) is a logically isolated network in AWS. Its IPv4 CIDR defines the address range from which its subnets are allocated. CIDR notation combines a network address with a prefix length; for example, a `/20` is smaller than a `/16`. The chosen subnet must be inside the VPC range and must not overlap any existing subnet in that VPC. Check the actual subnet list before choosing a range; do not reuse the example range from a screenshot without validating it.

3.2 Public and private subnets

A subnet is public when its effective route table has a route to an Internet Gateway. For an EC2 instance to be reachable directly over public IPv4, it also needs suitable public addressing and security/host rules. A private subnet has no direct route to an Internet Gateway. Private instances can still communicate over private IP addresses when the VPC routes, security groups, network ACLs, and operating-system firewall permit it.

3.3 Route tables and Internet Gateway

A route table selects a target for traffic based on its destination. A VPC route table normally includes a local route for the VPC CIDR. A public route table commonly has `0.0.0.0/0` targeting an Internet Gateway. For this exercise, the private subnet's associated route table should retain the local route and should not have a default route to an Internet Gateway. If private instances

need outbound internet access later, design egress separately, for example with a NAT Gateway or suitable VPC endpoints.

3.4 Security groups

Security groups are stateful virtual firewalls attached to network interfaces. SSH normally uses TCP port 22. The private RHEL security group should allow inbound TCP/22 from the public Ubuntu instance's security group. The public Ubuntu security group should allow inbound TCP/22 only from a trusted client IP/CIDR when possible. A route determines the path; a security group determines whether the traffic is permitted. Both must allow the intended connection.

3.5 SSH keys, Pageant, and agent forwarding

SSH public-key authentication uses a private key on the client and a matching public key listed in the target account's `authorized_keys` file. The private key must not be copied into that file or sent to the server. PuTTY is the Windows SSH client; PuTTYgen can convert/manage key formats; Pageant acts as an SSH authentication agent. Agent forwarding allows the remote jump host to request authentication operations from the client-side agent. Use it only with a host you trust: a compromised jump host may be able to request signatures while the forwarded agent is available.

3.6 Linux permissions and SELinux

The mode `700` on an SSH directory allows only its owner to access it. The mode `600` on `authorized_keys` allows only the owner to read and write the file. Ownership and mode bits are separate from SELinux mandatory access controls. On RHEL, `restorecon` restores default SELinux labels for a path; `ls -Z` displays those labels.

4 Target Architecture

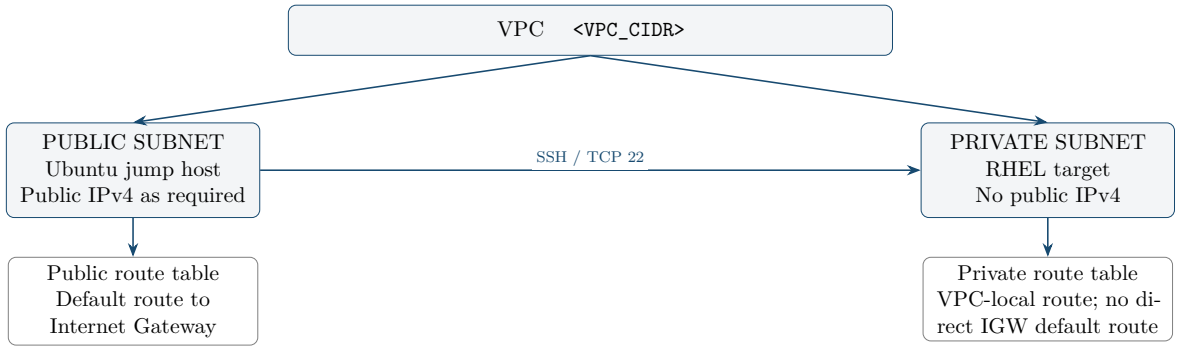


Figure 1: Logical topology. Confirm all labels and routes against the live AWS configuration.

5 Implementation Procedure

Follow this procedure in the same order as the lab was performed: first establish access to the public Ubuntu host, then prepare its custom login, then build the private network and target instance, and finally configure SSH access to RHEL. Do not proceed past a verification checkpoint

until the expected result is confirmed. Console labels may change slightly as AWS updates its interface.

5.1 Step 1: Launch and verify the public Ubuntu instance

Console path: AWS Console → EC2 → Instances → Launch instances

1. Select an Ubuntu Server AMI and an instance type suitable for the lab.
2. Select the intended VPC and a subnet that has a route to an Internet Gateway.
3. Enable public IPv4 assignment if required for direct PuTTY access.
4. Select or create a key pair and download/store the private key securely. Do not share it or place it in the report.
5. Configure the instance security group to allow SSH (TCP/22) from your current trusted public IP address or a narrow approved CIDR. Avoid opening SSH to everyone.
6. Launch the instance. Wait until it is running and its status checks pass.
7. Record the instance ID, VPC, subnet, private IP, and current public IPv4 address.

Why this step matters. This Ubuntu instance is the public jump host. It provides the first SSH entry point; the RHEL target will later be accessed through its private IP.

Verify before continuing. The Ubuntu instance is running, has passed status checks, and has a working public access path. Use its current public IP, not an old screenshot value.

5.2 Step 2: Connect through PuTTY as the default Ubuntu user

Console path: PuTTY on Windows

1. Open PuTTY and enter the Ubuntu instance's current public IPv4 address as the host, with port 22 and SSH selected.
2. Under SSH authentication credentials, select the private-key file downloaded for the AWS key pair. If PuTTY requires its own key format, use PuTTYgen to load the downloaded key and save a PuTTY-compatible private key (`.ppk`).

   NOTE: for reference, follow the procedure of 5.4.2 Convert the private key using PuTTYgen.

3. Under Connection → Data, enter the default Ubuntu AMI username, usually `ubuntu`.
4. Open the connection and verify the host key using a trusted source when connecting for the first time.
5. After login, run the following command:

```
whoami
```

Verify before continuing. The terminal prints `ubuntu`. This confirms the initial access path works before creating a custom account.

5.3 Step 3: Create a custom Ubuntu user

While logged in as the default `ubuntu` account, create the custom user. Use a lowercase username consistently throughout the lab.

```
sudo adduser <UBUNTU_USER>
id <UBUNTU_USER>
sudo mkdir -p /home/<UBUNTU_USER>/.ssh
sudo chown <UBUNTU_USER>:<UBUNTU_USER> /home/<UBUNTU_USER>/.ssh
sudo chmod 700 /home/<UBUNTU_USER>/.ssh
```

Follow the prompts to create the account. Use a strong password if requested; SSH key authentication will be configured next. Do not use a username that already belongs to another account.

Verify before continuing. The `id` command shows the new user, and the user's home directory and `.ssh` directory exist.

5.4 Step 4: Generate a key for the custom Ubuntu user and convert it for PuTTY

5.4.1 Generate the key pair on Windows

Open Windows PowerShell and generate a dedicated Ed25519 key pair. Replace the placeholder with a filename you control:

```
ssh-keygen -t ed25519 -f "$HOME\.ssh\<UBUNTU_KEY_NAME>"
```

Set a strong passphrase when prompted. The private key is the file without the `.pub` extension; the public key ends in `.pub`. Keep the private key secret.

5.4.2 Convert the private key using PuTTYgen

1. Open PuTTYgen and use **Load** to select the generated private key. If the file picker hides key files, choose the option to show all files.
2. If prompted, enter the key passphrase.
3. Save the key in PuTTY's private-key format (`.ppk`) using **Save private key**. Store it securely and keep its passphrase.
4. Copy the complete public-key line from the `.pub` file or from PuTTYgen's public-key field. Do not copy or upload the private key.

5.4.3 Install the public key for the new Ubuntu user

Return to the existing PuTTY session logged in as `ubuntu` and open the authorized-keys file for the new user:

```
sudo vi /home/<UBUNTU_USER>/.ssh/authorized_keys
```

In `vi`, press `i`, paste the complete public-key line as one continuous line, press `Esc`, type `:wq`, and press Enter. Then apply and verify ownership and permissions:

```
sudo chown <UBUNTU_USER>:<UBUNTU_USER> /home/<UBUNTU_USER>/.ssh
sudo chmod 700 /home/<UBUNTU_USER>/.ssh
sudo chown <UBUNTU_USER>:<UBUNTU_USER> /home/<UBUNTU_USER>/.ssh/authorized_keys
sudo chmod 600 /home/<UBUNTU_USER>/.ssh/authorized_keys
sudo ls -ld /home/<UBUNTU_USER>/.ssh
sudo ls -l /home/<UBUNTU_USER>/.ssh/authorized_keys
```

Verify before continuing. The SSH directory belongs to the custom user and has mode 700; `authorized_keys` belongs to that user and has mode 600. Only the public key is stored on Ubuntu.

5.5 Step 5: Log in as the custom Ubuntu user and configure Pageant

5.5.1 Test the custom account in PuTTY

Open a new PuTTY session or update the saved session. Use the current Ubuntu public IP, port 22, username `<UBUNTU_USER>`, and the matching `.ppk` private key. Connect and run:

```
whoami
```

NOTE: Refer 5.2 Step 2: Connect through PuTTY as the default Ubuntu user

Verify before continuing. The command prints the custom Ubuntu username, not `ubuntu`. If authentication fails, verify the username, key pair, public-key line, ownership, and file permissions before changing other settings.

Then to end the session run :

```
exit
```

5.5.2 Configure Pageant and agent forwarding

1. Start Pageant on Windows and add the original AWS private key needed for the next SSH hop. Enter its passphrase locally if requested.
2. In PuTTY Configuration, open Connection → SSH → Auth (or the equivalent authentication section in your version) and enable agent forwarding.
3. Save the session settings, reconnect to Ubuntu as the custom user, and keep Pageant running.
4. On Ubuntu, check whether the forwarded agent is available:

```
ssh-add -L
```

If the command cannot connect to an agent or does not list the expected identity, troubleshoot Pageant and PuTTY agent-forwarding settings before continuing.

Why this step matters. Agent forwarding lets the remote SSH client request authentication operations from the Windows-side agent; it does not copy the private key to Ubuntu. Only forward the agent through a host you trust.

Verify before continuing. The expected public-key identity is listed by `ssh-add -L`. A saved GUI checkbox alone is not proof that forwarding is working.

Use these screenshots as Reference:

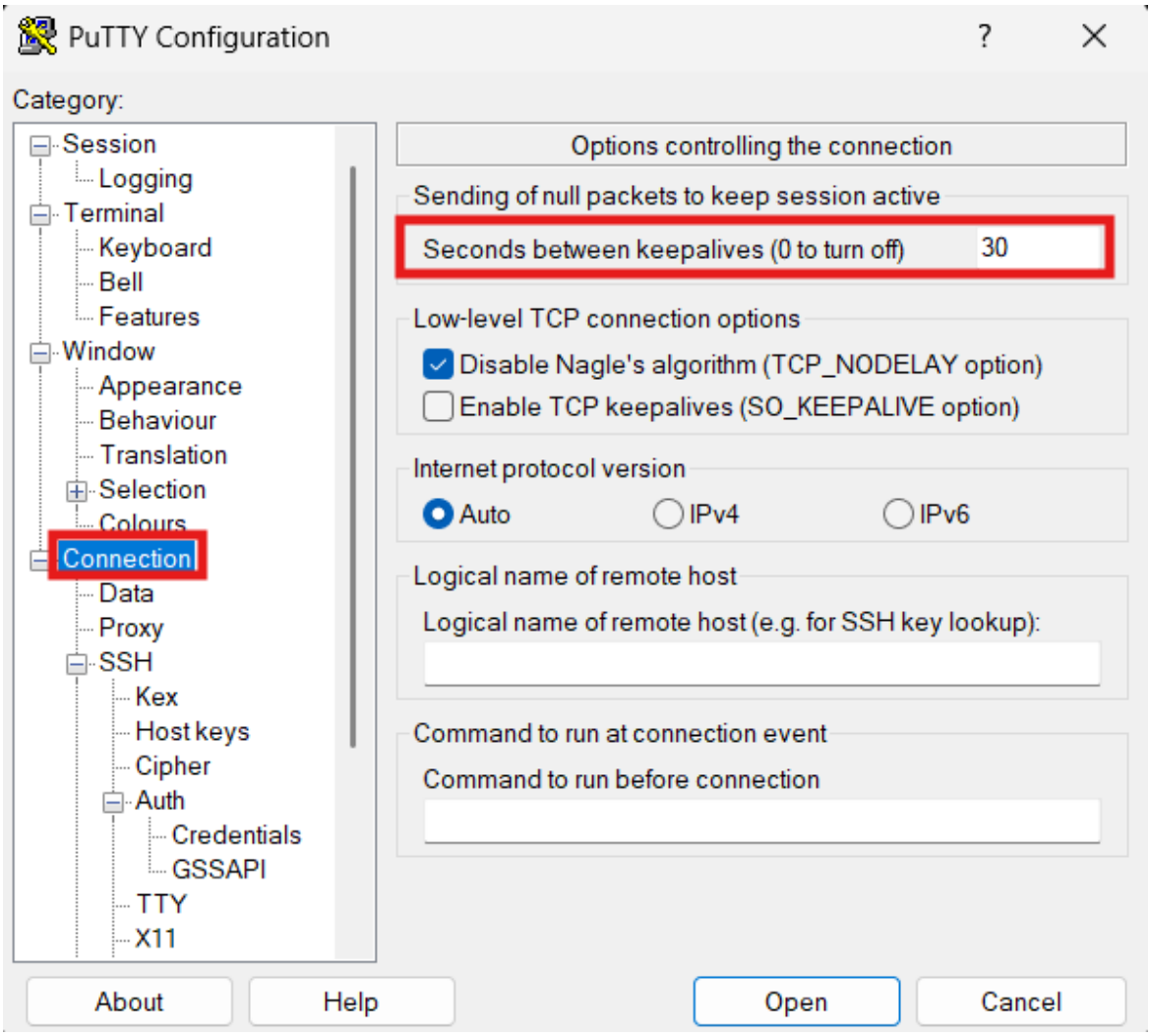


Figure 2: PuTTY connection settings. Confirm these settings against your own session.

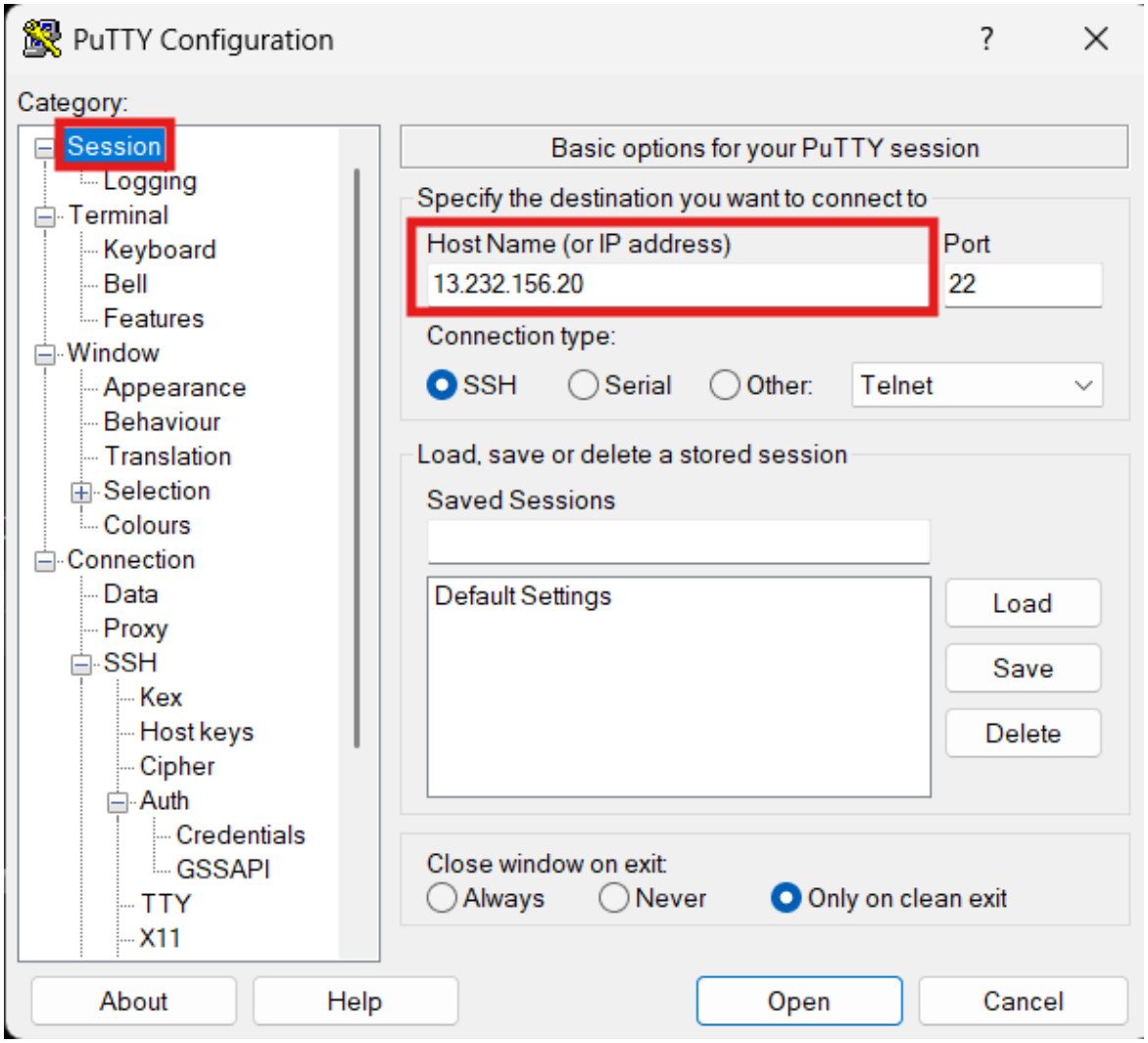


Figure 3: PuTTY Session page. Use the current jump-host address and SSH port.

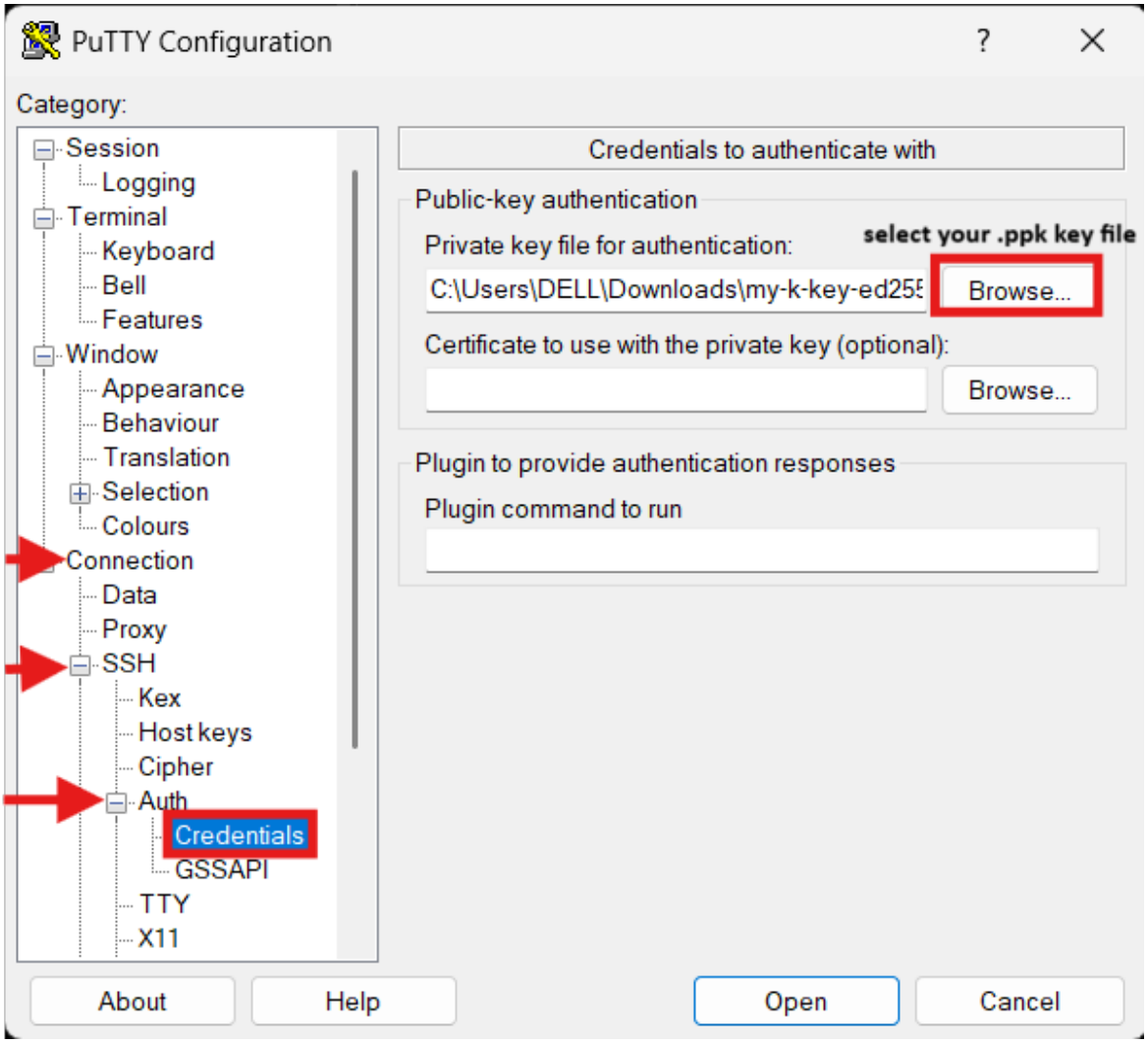


Figure 4: PuTTY authentication/key configuration.

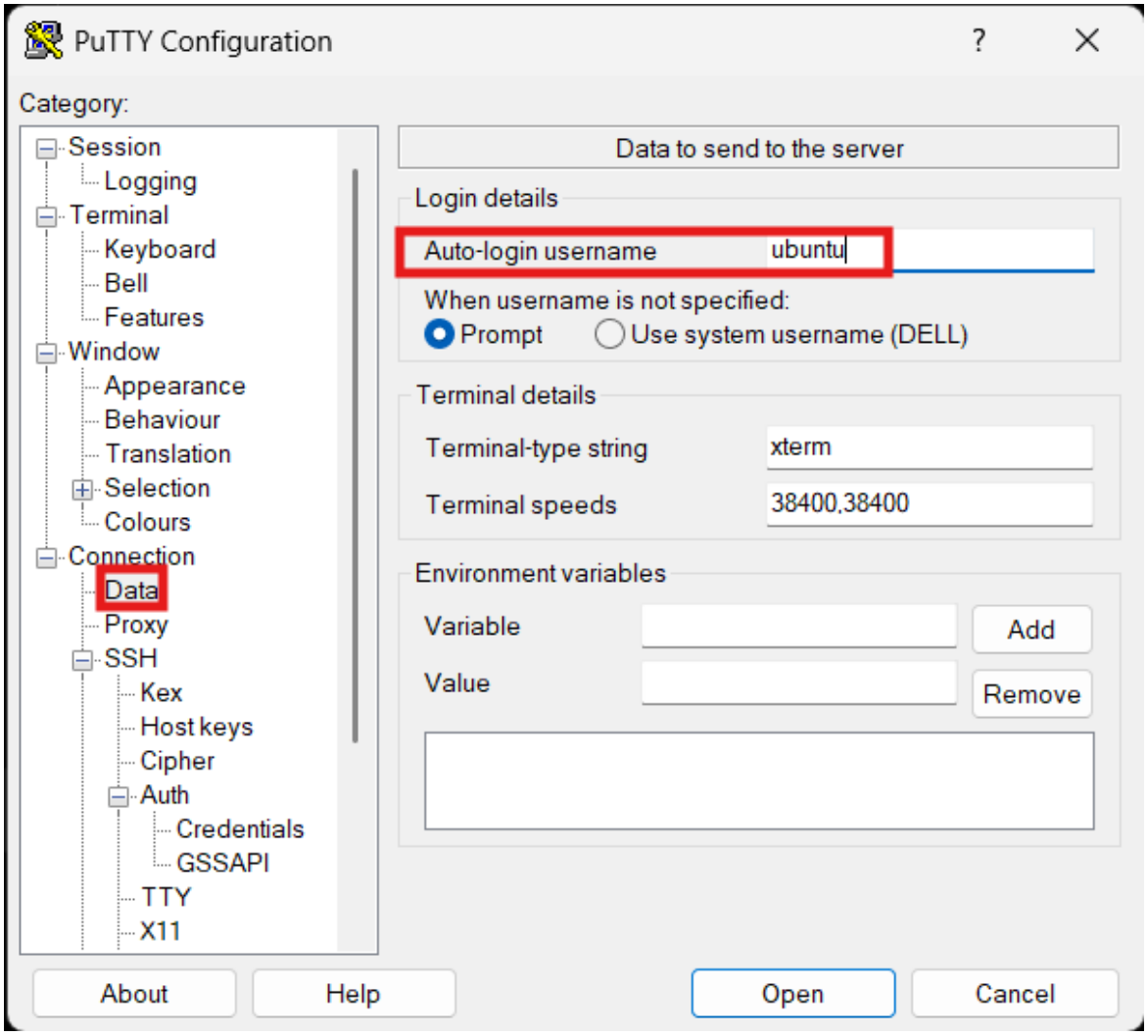


Figure 5: PuTTY connection data and username settings. If the screenshot shows the default `ubuntu` account, treat it as an earlier example; use the custom account configured in Step 6 for the verification test.



Figure 6: Pageant key list. The key name is specific to the example environment.

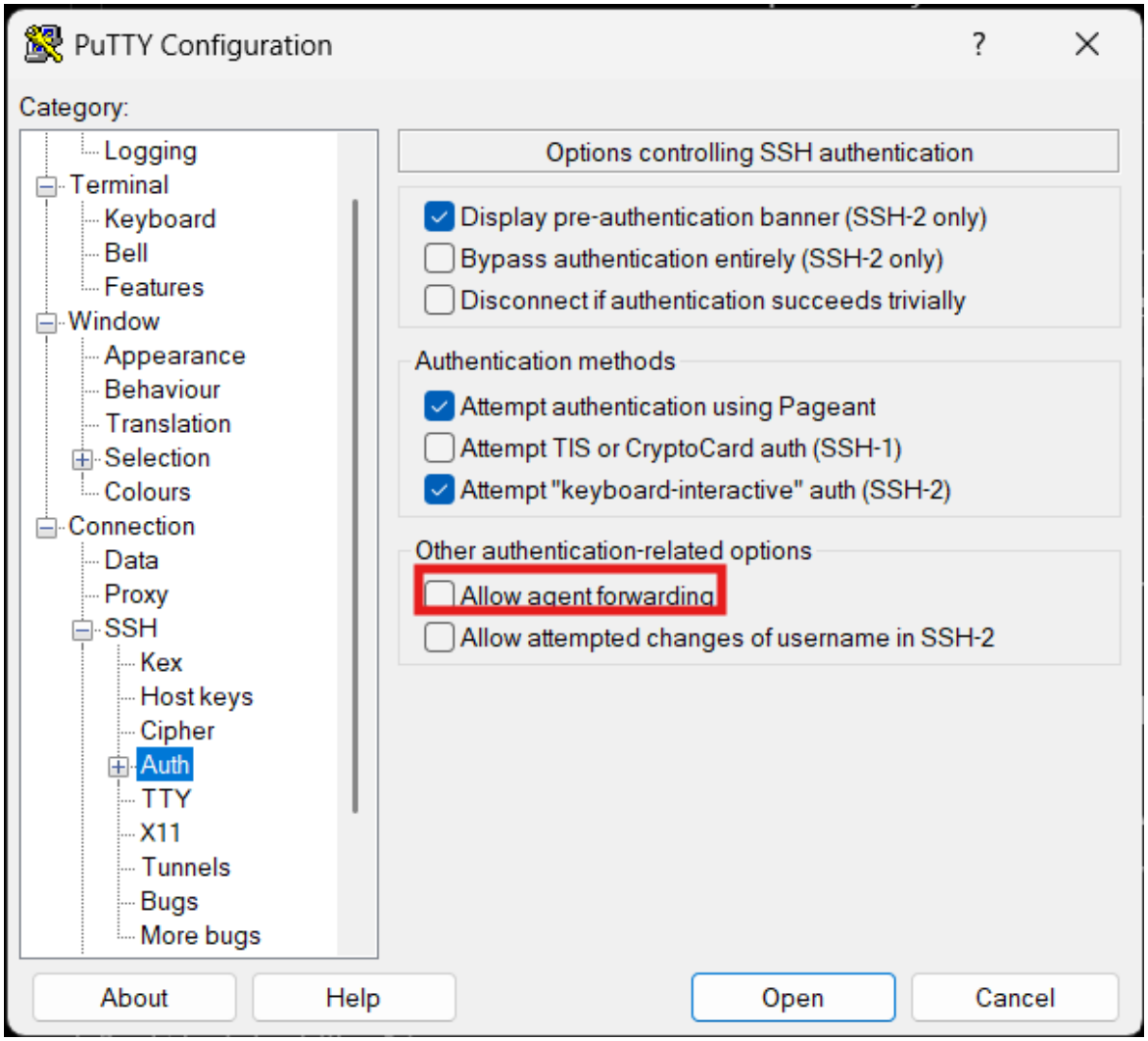


Figure 7: PuTTY SSH authentication settings. Verify the agent-forwarding option is enabled in the active session.

5.6 Step 6: Identify the VPC and record existing subnet CIDRs

Console path: AWS Console → VPC → Your VPCs

1. Confirm the VPC ID used by the Ubuntu instance by checking its EC2 instance details.
2. Record the VPC IPv4 CIDR and all existing subnet CIDRs.
3. Choose a private subnet CIDR that falls within the VPC range and does not overlap an existing subnet.

Why this step matters. The route table and private subnet must be created in the same VPC as the jump host so that the two instances can communicate over private addresses.

Verify before continuing. The intended VPC is confirmed and a valid non-overlapping subnet CIDR has been selected.

5.7 Step 7: Create the dedicated private route table

Console path: VPC → Route tables → Create route table

1. Create a route table in the same VPC. Name it `PRIVATE-ROUTE-TABLE`.
2. Open its Routes tab and confirm that the VPC-local route exists with target `local`.
3. Do not add a default route `0.0.0.0/0` to an Internet Gateway for this private route table.
4. The subnet association will be configured after the private subnet has been created.

Why this step matters. Creating the route table first lets the private subnet be associated with the intended table as soon as the subnet exists.

Verify before continuing. The dedicated route table exists in the correct VPC and contains the local route without a direct Internet Gateway default route.

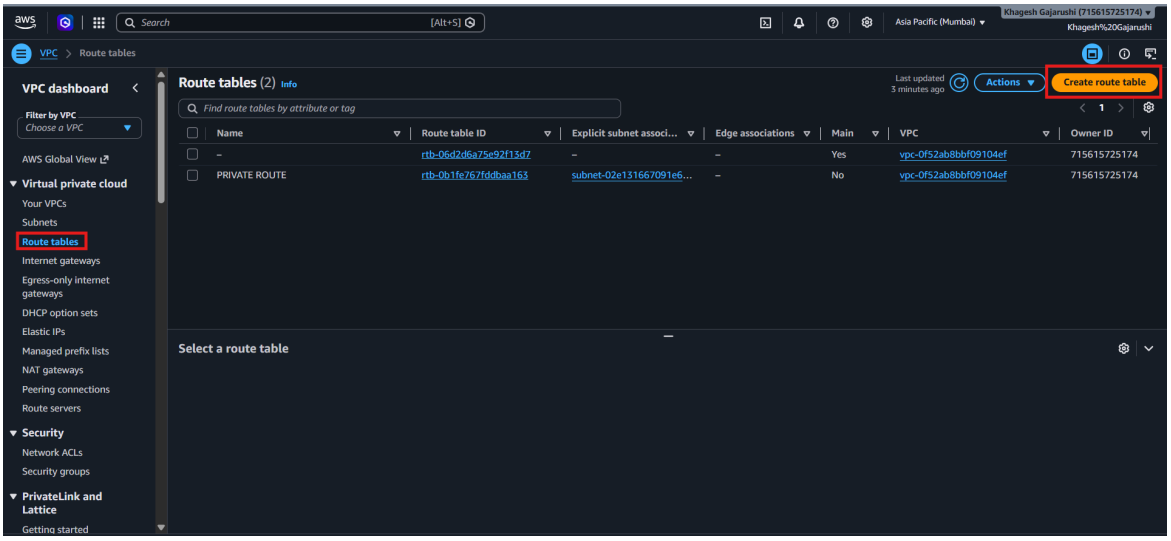


Figure 8: Route-table list and association area. Use the Routes tab to verify the actual routes.

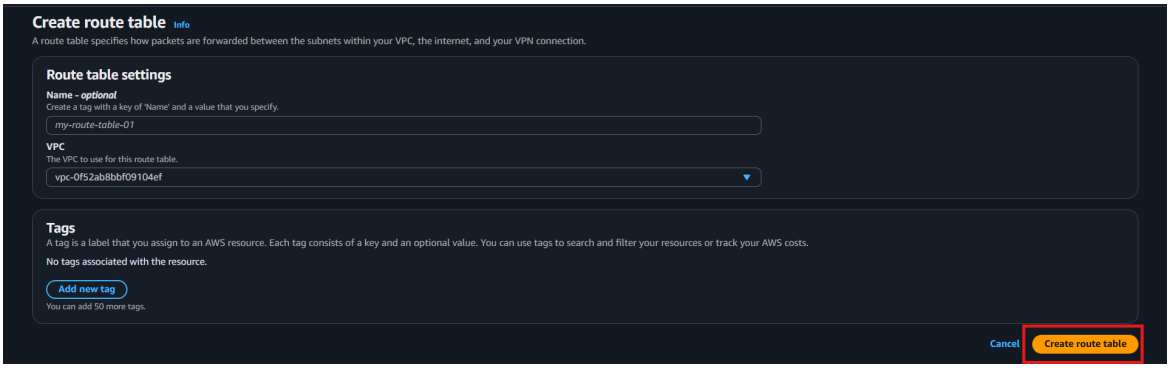


Figure 9: Create route table form, including the selected VPC.

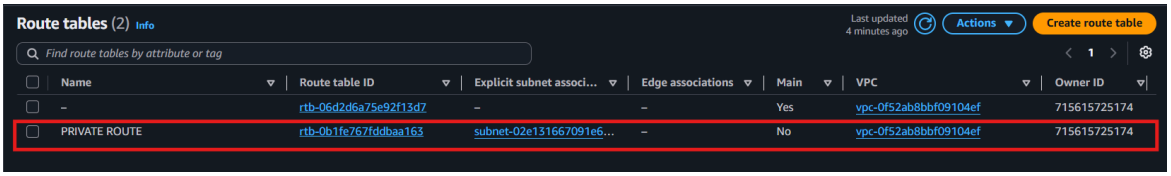


Figure 10: Route-table list after creation. Confirm route entries and subnet association in the live console.

5.8 Step 8: Create the private subnet and associate its route table

Console path: VPC → Subnets → Create subnet

1. Select the same VPC used by the Ubuntu instance and the private route table.
2. Enter a descriptive name, select the intended Availability Zone, and enter the validated non-overlapping private CIDR.
3. Create the subnet and verify its VPC, subnet ID, Availability Zone, and CIDR.
4. Return to Route tables, select the private route table created in Step 7, open **Subnet associations**, choose **Edit subnet associations**, select the new private subnet, and save.
5. Reopen the route table and confirm that the private subnet is explicitly associated with it.

Why this step matters. A route table affects a subnet only when associated with it. A private subnet's local route supports communication within the VPC; no direct Internet Gateway default route is added.

Verify before continuing. The private subnet is inside the VPC range, does not overlap another subnet, and is associated with the dedicated private route table.

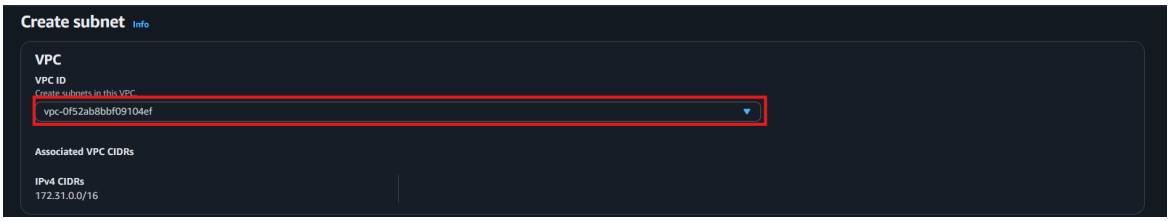


Figure 11: Subnet creation: VPC selection in the console.

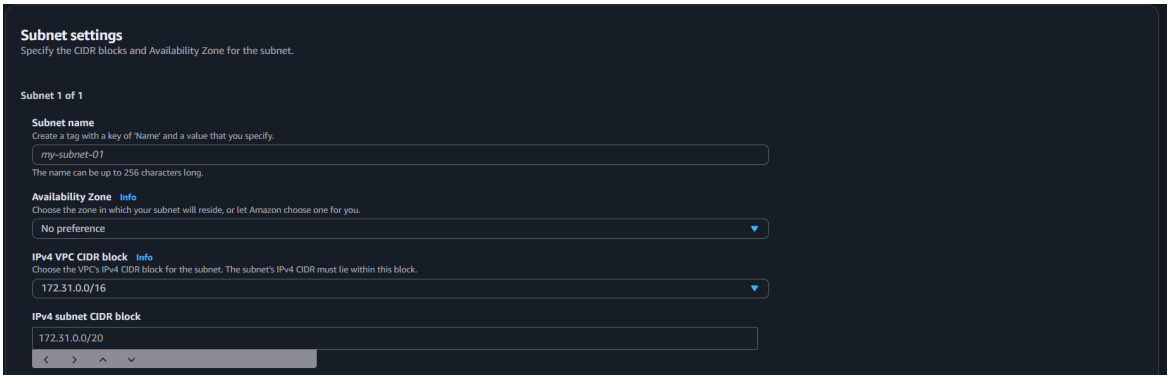


Figure 12: Subnet creation: name, Availability Zone, and CIDR settings.

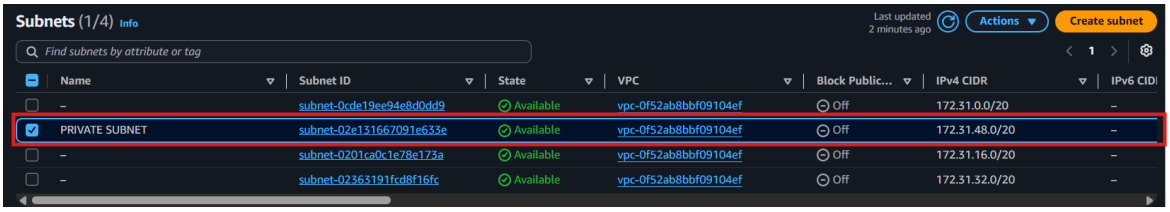


Figure 13: Subnet list: verify the created subnet and its CIDR.

5.9 Step 9: Configure security groups for both instances

Console path: EC2 → Network & Security → Security Groups

1. Identify the security group attached to the Ubuntu jump host by checking the instance's Security tab.
2. Identify or create a separate security group for the RHEL instance in the same VPC.
3. In the RHEL security group's inbound rules, allow SSH (TCP/22) with the source set to the Ubuntu jump host's security group.
4. In the Ubuntu security group's inbound rules, allow SSH only from your trusted client public IP/CIDR or another approved narrow source.
5. Confirm that the intended groups are attached to the correct instances. Do not use `0.0.0.0/0` as the RHEL SSH source.

Why this step matters. Routing determines the network path; security groups determine whether traffic is permitted. Both must be configured correctly.

Verify before continuing. Public SSH is restricted to the trusted client, and private SSH is allowed from the jump-host security group only.

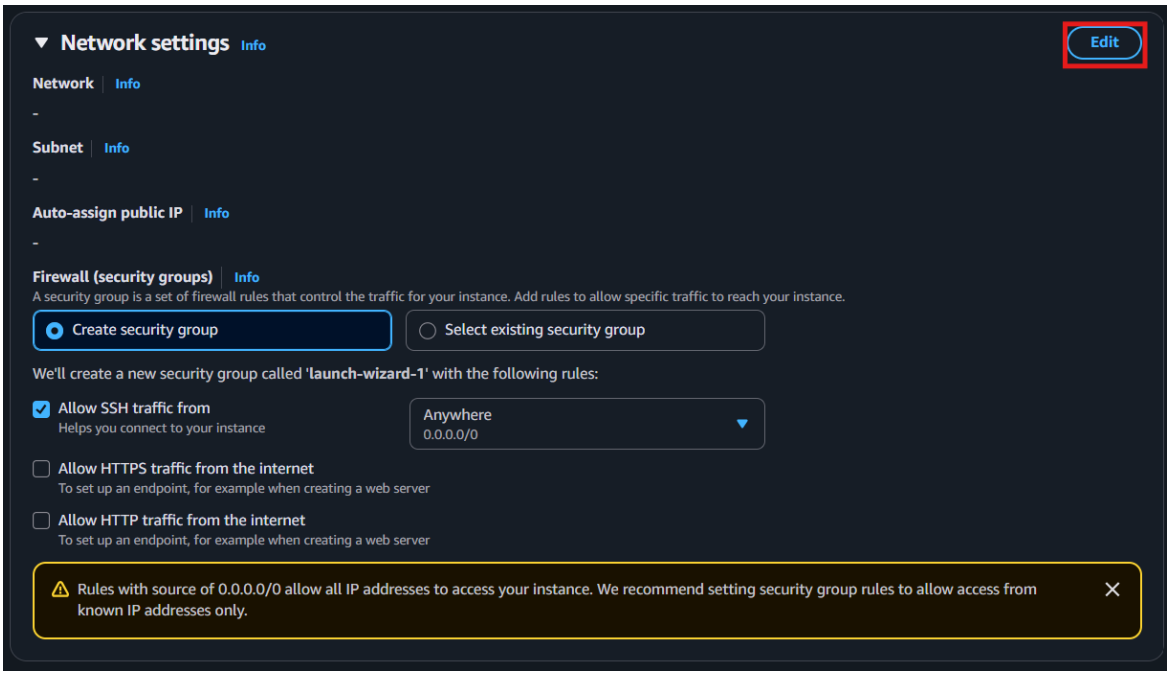


Figure 14: Earlier security-group/launch settings. If the image shows SSH from Anywhere, treat it as an example of a setting to review—not the desired final private-host rule.

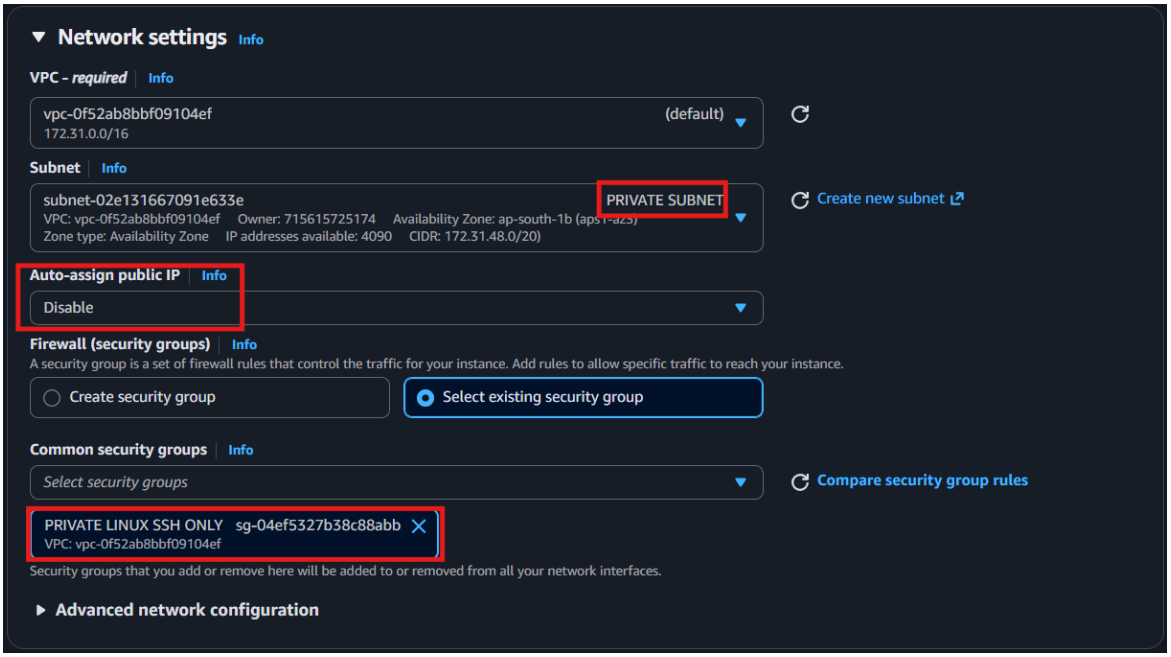


Figure 15: Private subnet and public-IP setting in the launch workflow. Verify the current instance settings in the console.

Important: Screenshots are examples, not proof of the live security-group state. Inspect the current inbound rules before testing or declaring the setup secure.

5.10 Step 10: Launch or configure the private RHEL instance

Console path: EC2 → Instances → Launch instances

1. Select the required RHEL AMI and a suitable instance type.
2. Choose the same VPC as Ubuntu and select the private subnet created in Step 8.
3. Disable automatic public IPv4 assignment if that option is available in the launch workflow; verify the instance has no public IPv4 address after launch.
4. Attach the RHEL security group configured in Step 9.
5. Launch the instance and wait for it to become running with passing status checks.
6. Record its private IPv4 address and confirm its subnet is associated with the dedicated private route table.

Why this step matters. The RHEL instance is intended to be reached through the jump host, not directly from the public internet. A private IP alone is not enough; routing and security-group rules must also be correct.

Verify before continuing. RHEL is running in the intended private subnet, has no public IPv4 address, and uses the intended security group.

5.11 Step 11: Test SSH access from Ubuntu to RHEL

Connect to Ubuntu as the custom Ubuntu user, then run the following commands using the RHEL private IP and the initial authorized RHEL account (commonly `ec2-user` for the relevant RHEL AMI):

```
whoami
nc -zv -w 3 <RHEL_PRIVATE_IP> 22
ssh -v <RHEL_ADMIN_USER>@<RHEL_PRIVATE_IP>
```

If `nc` is not installed, proceed with the SSH test while checking the security groups and host firewall. At first connection, verify the server host-key fingerprint through a trusted channel. An administrator on RHEL can obtain the Ed25519 host-key fingerprint with:

```
sudo ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```

Compare the fingerprint with the SSH client's prompt using a trusted channel. If that file does not exist, inspect the host keys available on the server rather than accepting an unverified key. After login, run:

```
whoami
hostname
```

Verify before continuing. TCP/22 is reachable and SSH authenticates to the intended RHEL administrative account. A failed ping alone does not prove SSH is broken because ICMP and TCP can be filtered differently.

5.12 Step 12: Create the custom RHEL user

While connected to RHEL through the authorized administrative account:

```
sudo useradd -m <RHEL_USER>
sudo passwd <RHEL_USER>
id <RHEL_USER>
sudo mkdir -p /home/<RHEL_USER>/.ssh
sudo chown <RHEL_USER>:<RHEL_USER> /home/<RHEL_USER>/.ssh
sudo chmod 700 /home/<RHEL_USER>/.ssh
sudo ls -ld /home/<RHEL_USER> /home/<RHEL_USER>/.ssh
```

Choose a strong, unique password if required by local account policy. If the account already exists, do not run `useradd` again; verify it with `id` and continue with the SSH-directory setup.

Verify before continuing. The user and home directory exist, and the SSH directory belongs to the target user with mode 700.

5.13 Step 13: Generate a dedicated SSH key pair on Ubuntu

For this lab workflow, generate a separate key pair on Ubuntu while logged in as the custom Ubuntu user. This is a lab convenience, not the preferred production design.

```
ssh-keygen -t ed25519 -f ~/rhel-user-key
ls -l ~/rhel-user-key*
```

Set a passphrase. The private key is `/rhel-user-key`; the public key is `/rhel-user-key.pub`.

The private key stays on Ubuntu for this lab flow; copy only the public-key line to the RHEL account.

Important: Generating a private key on the jump host is a lab compromise. In production, prefer keeping the private key on a trusted client or using an appropriately configured managed access design. Never place a private key in `authorized_keys`.

5.14 Step 14: Copy the public key into the RHEL user's authorized keys

On Ubuntu, display the public key:

```
cat ~/rhel-user-key.pub
```

Copy the entire single line beginning with `ssh-ed25519`. In the authenticated RHEL administrative session, prepare the authorized-keys file:

```
sudo touch /home/<RHEL_USER>/.ssh/authorized_keys
sudo chown <RHEL_USER>:<RHEL_USER> /home/<RHEL_USER>/.ssh/authorized_keys
sudo chmod 600 /home/<RHEL_USER>/.ssh/authorized_keys
sudo vi /home/<RHEL_USER>/.ssh/authorized_keys
```

In `vi`, press `i`, paste the public key as one continuous line, press `Esc`, type `:wq`, and press Enter. Then apply ownership, permissions, and SELinux labels:

```
sudo chown <RHEL_USER>:<RHEL_USER> /home/<RHEL_USER>/.ssh
sudo chmod 700 /home/<RHEL_USER>/.ssh
sudo chown <RHEL_USER>:<RHEL_USER> /home/<RHEL_USER>/.ssh/authorized_keys
sudo chmod 600 /home/<RHEL_USER>/.ssh/authorized_keys
sudo restorecon -RFv /home/<RHEL_USER>/.ssh
sudo ls -ldZ /home/<RHEL_USER>/.ssh
sudo ls -lZ /home/<RHEL_USER>/.ssh/authorized_keys
```

Verify before continuing. The SSH directory and file have the intended owner and modes, and SELinux labels match the system's expected SSH-file labels. Investigate unexpected labels rather than disabling SELinux.

5.15 Step 15: Verify the final login as the custom RHEL user

Return to Ubuntu and connect to RHEL using the dedicated private key:

```
ssh -i ~/rhel-user-key <RHEL_USER>@<RHEL_PRIVATE_IP>
```

Enter the key passphrase if requested. After login, run:

```
whoami
hostname
```

Verify before continuing. The username matches `<RHEL_USER>` and the hostname identifies the intended private RHEL instance. This is the final confirmation that the complete jump-host-to-private-instance path works.

6 Verification and Troubleshooting

6.1 Final verification checklist

Verification item	Status
Both instances are running and status checks pass	<PASS/FAIL>
Both instances belong to the intended VPC	<PASS/FAIL>
Private subnet CIDR is valid and non-overlapping	<PASS/FAIL>
Private subnet uses the dedicated private route table	<PASS/FAIL>
Private route table has local route and no direct IGW default route	<PASS/FAIL>
Private RHEL has no public IPv4 address	<PASS/FAIL>
Public SSH is limited to a trusted client source	<PASS/FAIL>
Private SSH source is the jump host security group	<PASS/FAIL>
Custom Ubuntu key-based login succeeds	<PASS/FAIL>
Agent identity is visible when forwarding is intended	<PASS/FAIL>
Host-key fingerprint was verified through a trusted channel	<PASS/FAIL>
Initial administrative SSH to RHEL succeeds	<PASS/FAIL>
Custom RHEL key-based login succeeds	<PASS/FAIL>
Ownership, modes, and SELinux contexts are verified	<PASS/FAIL>
Host-key fingerprint verified through a trusted channel	<PASS/FAIL>

6.2 Troubleshooting sequence

When SSH fails, troubleshoot in this order rather than changing several settings at once:

1. **Confirm the destination.** Check the current public IP for the jump host and the private IP for RHEL. Old public IPv4 addresses may no longer be assigned after stop/start.
2. **Confirm instance health and subnet association.** Both instances should be running and pass status checks; RHEL should be in the intended private subnet.
3. **Check routing.** Confirm the private subnet is associated with the intended route table and that the VPC-local route exists.
4. **Check security groups.** Confirm the public group permits the trusted client and the private group permits TCP/22 from the jump host's group.
5. **Check SSH service and host firewall.** On RHEL, check the daemon and listening socket:

   ```
   sudo systemctl status sshd
   sudo ss -lntp | grep ':22'
   ```

   A permitted network path is not enough if the SSH daemon is stopped or the operating-system firewall blocks the connection. A missing `grep` result is a clue to investigate, not a complete diagnosis.
6. **Check identity and key authorization.** Verify the exact username, that the public key is in the correct user's `authorized_keys`, and that the matching private key is being used.
7. **Check permissions and SELinux.** Verify directory/file ownership and modes, then inspect SELinux labels with `ls -Z`. Use `restorecon` rather than disabling SELinux as a first response.

8. **Interpret tests correctly.** Ping uses ICMP; SSH uses TCP/22. Ping failure alone does not prove TCP/22 is unreachable.
9. **Use verbose SSH output.** Run `ssh -v USER@HOST` to see whether the client reaches the server and whether a key is offered or accepted. Do not share logs that expose sensitive environment details.

7 Security Review and Cleanup

1. Restrict public SSH ingress to a trusted source and review the rule after the exercise.
2. Keep the private RHEL instance without public IPv4 addressing and allow inbound SSH only from the jump host security group.
3. Treat agent forwarding as a trust decision. Disconnect from untrusted hosts and do not forward an agent through a host you do not control.
4. Keep private keys out of screenshots, documentation, source control, and shared folders. Protect key files with an appropriate passphrase and operating-system permissions.
5. Review whether the custom RHEL private key should be rotated or removed after the lab, especially because this exercise creates it on the jump host.
6. Record the actual final configuration and distinguish observed results from expected results. Never claim a check passed unless you ran it.
7. Stop or terminate lab resources when they are no longer needed, after confirming that doing so will not affect other workloads. Review storage and other billable resources separately.

8 Conclusion

This exercise builds a basic controlled-access pattern: a public Ubuntu instance serves as the administrative jump host, while a RHEL instance remains on a private subnet without a public IPv4 address. The route table and security groups define the intended network path, and SSH public-key authentication provides access to Linux accounts. Verification of usernames, hostnames, key authorization, file permissions, and SELinux contexts helps establish that the connection reaches the intended system under the intended account.

The procedure is reusable only when its placeholders are replaced with values validated in the learner's own AWS environment. The supplied screenshots are evidence of one example workflow and should not override live configuration checks.

A Command Reference

Command	Purpose
`whoami`	Print the current effective username.
`hostname`	Print the host name of the system.
`id USER`	Display a user's UID, primary GID, and groups.
`ssh-keygen -t ed25519 -f PATH`	Generate an Ed25519 key pair.
`ssh-add -L`	List public keys available from the current SSH agent.
`ssh -v USER@HOST`	Connect with verbose SSH diagnostics.
`ssh -i KEY USER@HOST`	Connect using the specified private-key file.
`chmod 700 DIR`	Give the owner full directory access and deny group/other access.
`chmod 600 FILE`	Give the owner read/write file access and deny group/other access.
`chown USER:GROUP PATH`	Change file or directory ownership.
`restorecon -RFv PATH`	Restore expected SELinux contexts recursively and report changes.
`ls -ldZ PATH`	Display directory metadata and SELinux context.
`ls -lZ FILE`	Display file metadata and SELinux context.